6th International Conference on Artificial Intelligence and Robotics (MIRG-ICAIR 2026)

Navigating the Shielding Paradox

Runtime Shared Autonomy for Safe Learning-Based Mobile Robot Navigation

Sunday Ikechukwu

Department of Mechatronics Engineering, University of Benin, Nigeria

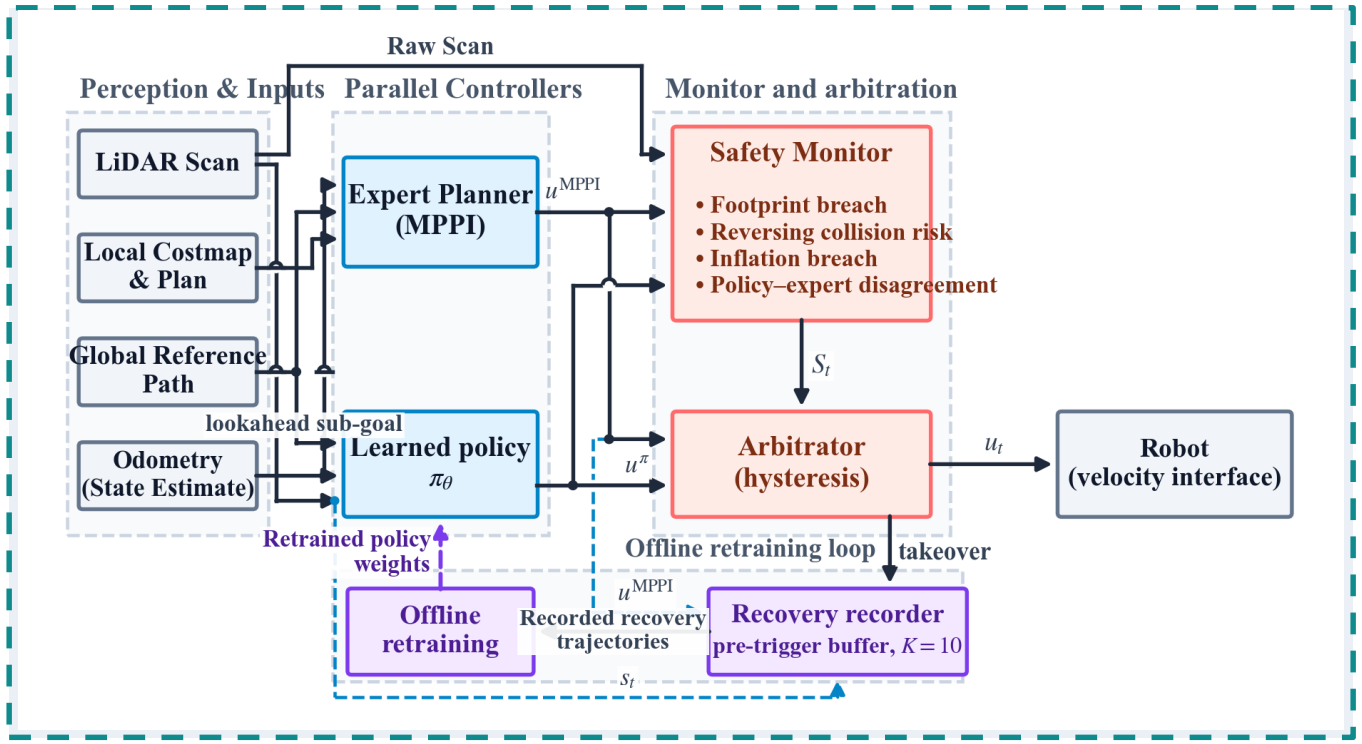
Central Industrial Liaison & Placement Unit (CILPU), University of Lagos, Nigeria

The problem

- Behavior cloning (BC) policies can enter states poorly represented in their training data (covariate shift), leading to navigation failures
- An expert planner(MPPI) can step in and help the robot recover
- However the expert itself can get stuck in narrow or cluttered geometries

Shielding paradox: intervention can support navigation, yet the expert planner can itself stall and limit task completion

A Learned Policy with an Expert Controller on Standby



1. **Learned policy**
   BC policy; primary velocity command
2. **Expert controller**
   Nav2 MPPI, running continuously as a hot standby
3. **Runtime monitor**
   Checks geometric risk and policy–expert disagreement
4. **Arbitrator**
   Selects which command is sent to the robot
5. **Recovery recorder**
   Keeps expert-labeled takeover data for retraining

Monitor conditions

- Footprint breach
- Reversing collision risk
- Costmap-inflation proximity
- Policy–expert disagreement

Both controllers are autonomous. Shared autonomy here means runtime control allocation between BC and MPPI

Interventions Also Become Training Data

1 Runtime takeover

- A monitored condition is met
- Control transfers from the learned policy to MPPI

2 Recovery-data collection

- Recorder keeps the preceding 10 control steps (1.0 s wall-clock)
- Keeps recording while MPPI is in control
- Each observation paired with its MPPI command

3 Policy retraining

- Recovery data aggregated with the initial expert demonstrations
- BC retrained iteratively
- Two recovery rounds give the final policy π_2

22,343
demonstration steps (133 episodes)

+ 10,724
recovery round 1

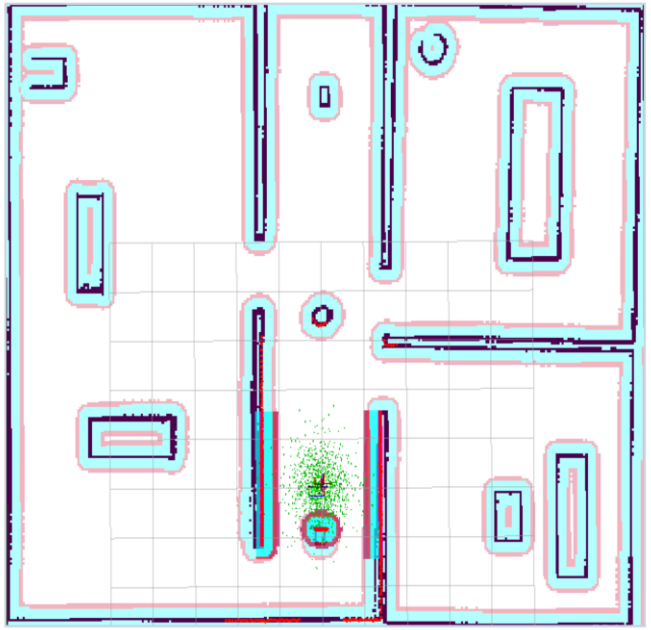
+ 22,049
recovery round 2

= 55,116
steps used to train final policy π_2

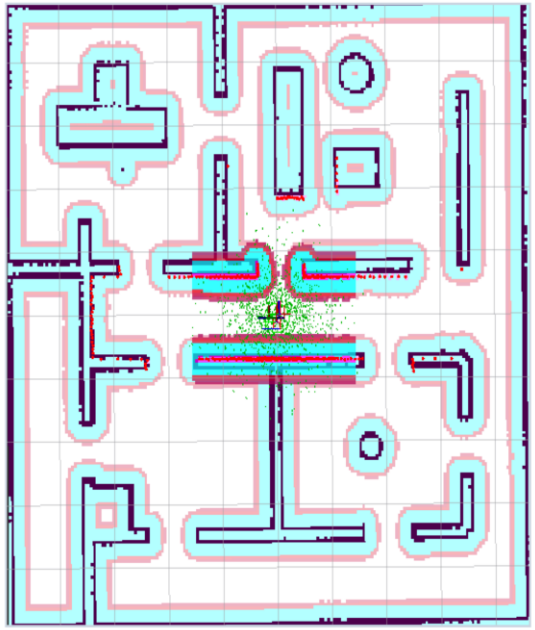
Not a new training algorithm: a recovery-centric data-collection and policy-improvement pipeline built around behavior cloning

Evaluation in Known and Unknown(Unseen) Indoor Environments

(a) Environment A (Known)



(b) Environment B (Unknown)



Platform (simulation only)

- Simulated TurtleBot3 Waffle
- Gazebo Classic
- ROS 2 Humble and Nav2
- Static obstacles only
- 8 scenarios per environment, one run each (n = 1)

Environment A (known)

Office-like layout: straight corridors, L-shaped corners, sparse obstacles. Demonstrations and recovery data were collected here.

Environment B (unseen)

Interconnected corridors, blind corners, offset doorways, cluttered furniture. No data from here used for training.

Configurations compared

- Initial BC (π_0)
- Recovery-trained BC (π_1, π_2)
- Standalone MPPI expert
- Monitored: π_2 + MPPI

Recovery Improved BC, but the Expert Fallback Exposed a Trade-off

Completed scenarios	Known (A)	Unseen (B)
Standalone policies and expert (no monitor)		
Initial BC (π_0)	1/8	1/8
Final RC-BC (π_2)	**8/8**	**6/8**
Standalone MPPI	8/8	6/8
Monitored system		
Monitored π_2 + MPPI	**8/8**	**4/8**

Collision events, unseen environment (standalone)

22	23	6
Final RC-BC	Initial BC	MPPI

What the results show

- Final RC-BC matches MPPI's completion counts: 8/8 known, 6/8 unseen
- Unseen: RC-BC had 22 collision events versus 6 for MPPI, at the same 6/8
- Monitored system: 8/8 known, 4/8 unseen; the other 4 ended after expert-planner deadlocks
- MPPI alone also aborted in B6 and B8: costmap constraints can trap the expert

Conclusion: the study shows both the value of recovery-centric policy improvement and the limits of a classical planner as a runtime fallback in constrained environments

Limits: simulation only · static obstacles · one run per scenario · no full-run collision totals for the monitored system